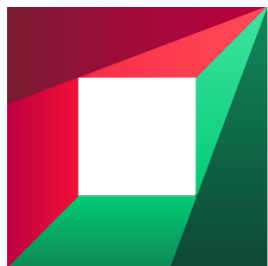


**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»**

КАФЕДРА ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ



**НГТУ
НЭТИ**

**Лабораторная работа №4
по дисциплине «ТРПО-2»
на тему
«Модульное тестирование. Разработка тестового
покрытия.
Тестирование производительности»**

Группа:

Факультет: АВТФ

Студенты:

Преподаватель: Романов Е.Л.

Новосибирск 2022

Оглавление

1	Задание на лабораторную работу.....	3
2	Ход работы	3
2.1.	Разработка тестового покрытия для модульных тестов.....	3
2.2.	Тестирование производительности метода сортировки	7
	Заключение	17
	Приложение А	17

1 Задание на лабораторную работу

1. Разработка тестового покрытия для модульных тестов.

Разработать модульные тесты для заданного метода балансировки, реализованного в л.р. 1-3.

2. Выполнить тестирование производительности метода балансировки.

Оценить вид зависимости трудоемкости и «грязного» времени работы метода от размерности СД.

2 Ход работы

2.1. Разработка тестового покрытия для модульных тестов

Модульные тесты должны проверять корректность исполнения балансировки для перечисленных тестовых случаев:

- Значения добавлены в дерево в строго возрастающем порядке (тест 1);
- Значения добавлены в дерево в строго убывающем порядке (тест 2);
- Исходный набор содержит одинаковые значения (тест 3);
- Дерево уже сбалансировано (тест 4);
- Несбалансированное дерево имеет две ветви (со строго возрастающими и строго убывающими значениями относительно корня) (тест 5);
- Исходные данные неупорядоченные (на входе) (тест 6);
- Элемент из середины интервала встречается несколько раз (тест 7);
- Элемент из середины интервала встречается в листе левого поддерева (тест 8);
- Элемент из середины интервала встречается в корне дерева (тест 9).

В качестве параметра сбалансированности возьмём сумму значений длин путей в дереве. Актуальное значение суммы длин путей получается с

помощью добавленного метода (листинг 1) в класс структуры данных – *getSumPathLengths()* (за счёт рекурсивного обхода дерева).

```
private int getSum(int num, int level){
    //передаём левел с передачей рекурсивного значения
    if (num >= size || arrayTree.get(num) == null)
        return 0;
    return level + getSum(2 * num + 1, level + 1) +
getSum(2 * num + 2, level + 1);
}

public int getSumPathLengths(){
    return getSum(0, 0);
}
```

Листинг 1. Метод getSumPathLengths()

Ожидаемое значение получается следующим образом:

1. В актуальное дерево добавляются 2^N-1 вершин, где N – максимальный уровень дерева (после балансировки);
2. Таким образом, после балансировки данные вершины заполняют все узлы в дереве до N -ого уровня;
3. При этом сумма длин путей для такого дерева будет иметь чёткую зависимость от N уровней. Алгоритм получения ожидаемого значения суммы длин путей сбалансированного дерева представлен в листинге 2.

```
private int getExpectedSumPathLength(int level){
    int sum = 0;
    for (int curLevel = 0, N=1; curLevel < level;
curLevel++, N*=2)
        sum+=curLevel*N;

    return sum;
}
```

*Листинг 2. Алгоритм получения ожидаемого значения суммы длин путей
дерева*

Код модульных тестов на JUnit приведён в Приложении А. Результат тестирования модульными тестами JUnit приведён на рис. 1 – 5

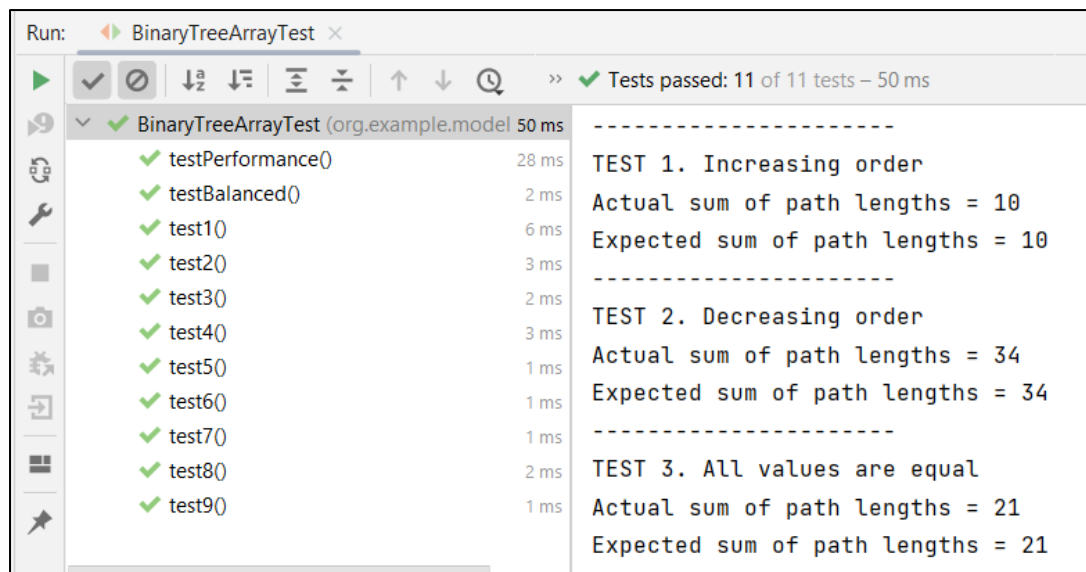


Рисунок 1. Результат выполнения модульного тестирования

TEST 1. Increasing order
Actual sum of path lengths = 10
Expected sum of path lengths = 10

Рисунок 2. Результаты теста 1

TEST 2. Decreasing order
Actual sum of path lengths = 34
Expected sum of path lengths = 34

Рисунок 3. Результаты теста 2

TEST 3. All values are equal
Actual sum of path lengths = 21
Expected sum of path lengths = 21

Рисунок 4. Результаты теста 3

TEST 4. Tree is balanced already
Actual sum of path lengths = 258
Expected sum of path lengths = 258

Рисунок 5. Результаты теста 4

TEST 5. Two branches
Actual sum of path lengths = 10
Expected sum of path lengths = 10

Рисунок 6. Результаты теста 5

TEST 6. Values isn't ordered
Actual sum of path lengths = 98
Expected sum of path lengths = 98

Рисунок 7. Результаты теста 6

TEST 7. The middle element is repeated several times
Actual sum of path lengths = 11
Expected sum of path lengths = 11

Рисунок 8. Результаты теста 7

TEST 8. The middle element in leaf of left subtree
Actual sum of path lengths = 10
Expected sum of path lengths = 10

Рисунок 9. Результаты теста 8

TEST 9. The middle element in root of tree
Actual sum of path lengths = 10
Expected sum of path lengths = 10

Рисунок 10. Результаты теста 9

Внесём дефект в код метода балансировки (листинг 3) и посмотрим результаты выполнения модульного тестирования (рис. 11 – 20).

```
//Балансировка дерева
public BinaryTreeArray balance(){

    int sz1=getSize(1); // ДЕФЕКТ: было int
    sz1=getSize(0);

    Vector <Object> newArray = new Vector<Object> (size);
    //вектор индексов
```

```

        newSizeBalance = 10;
        ArrayList<Object> newArrayTree = new
ArrayList<Object>(size);
        for(int i = 0; i < size; i++) {
            newArrayTree.add(null);
        }
        set(newArray,0);
        balance(newArray,0, sz1, newArrayTree);
        int curMaxLevel = 0;
        if (this.maxLevel != 0)
            if (this.maxLevel == 1)
                curMaxLevel = 1;
            else
                curMaxLevel = log2Count(countObject);
        else
            curMaxLevel = 0;
        BinaryTreeArray balanced = new
        BinaryTreeArray(newSizeBalance, newArrayTree, this.comparator,
        this.countObject,
            curMaxLevel);
        return balanced;
    }

```

Листинг 3. Метод балансировки дерева с внесённым дефектом

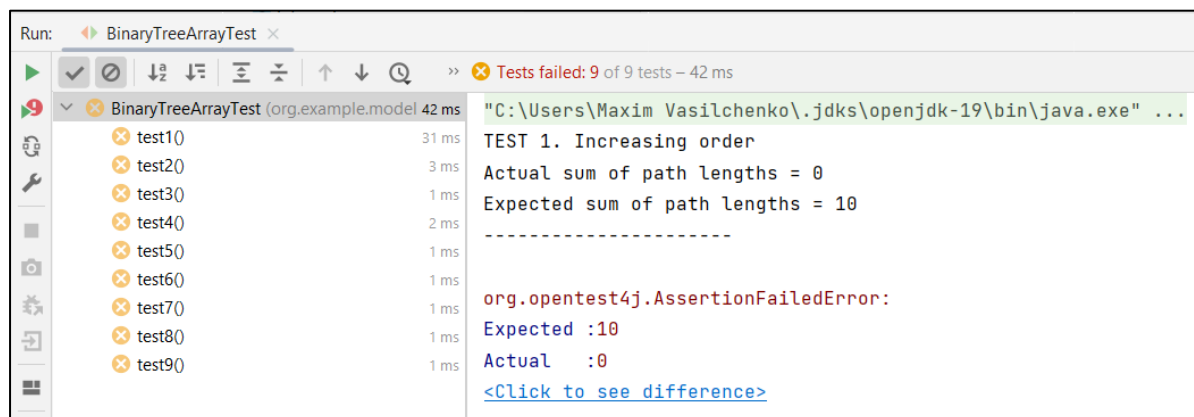


Рисунок 11. Результат выполнения модульного тестирования

TEST 1. Increasing order
Actual sum of path lengths = 0
Expected sum of path lengths = 10

TEST 2. Decreasing order
Actual sum of path lengths = 31
Expected sum of path lengths = 34

TEST 3. All values are equal
Actual sum of path lengths = 0
Expected sum of path lengths = 21

Рисунок 12. Результаты теста 1 (дефект)

Рисунок 13. Результаты теста 2 (дефект)

Рисунок 14. Результаты теста 3 (дефект)

TEST 4. Tree is balanced already
Actual sum of path lengths = 0
Expected sum of path lengths = 258

Рисунок 15. Результаты теста 4 (дефект)

TEST 7. The middle element is repeated several times
Actual sum of path lengths = 6
Expected sum of path lengths = 11

Рисунок 18. Результаты теста 7 (дефект)

TEST 5. Two branches
Actual sum of path lengths = 2
Expected sum of path lengths = 10

Рисунок 16. Результаты теста 5 (дефект)

TEST 8. The middle element in leaf of left subtree
Actual sum of path lengths = 6
Expected sum of path lengths = 10

Рисунок 19. Результаты теста 8 (дефект)

TEST 6. Values isn't ordered
Actual sum of path lengths = 90
Expected sum of path lengths = 98

Рисунок 17. Результаты теста 6 (дефект)

TEST 9. The middle element in root of tree
Actual sum of path lengths = 2
Expected sum of path lengths = 10

Рисунок 20. Результаты теста 9 (дефект)

2.2. Тестирование производительности метода балансировки

Для выполнения экспериментов по тестированию производительности метода балансировки были реализованы и добавлены (изменены) следующие части программного кода:

1. В класс структуры данных «Бинарное дерево в массиве» добавилось поле *int maxLevel* – максимальная длина ветви дерева.
2. В класс структуры данных «Бинарное дерево в массиве» добавилось поле *int countObject* – общее количество хранимых объектов в дереве.
3. Добавилось перечисление *DebugLevel* для отображения в лог (консоль) дополнительной информации о вставке значений в дерево. Возможные режимы:
 - a. *DebugLevel.OFF* – не отображать в консоль дополнительную информацию;
 - b. *DebugLevel.INCREASE_ARRAYLIST* – отображать дополнительную информацию в консоль в момент увеличения размерности содержащего дерево динамического массива;
 - c. *DebugLevel.CHANGE_MAXLEVEL* – отображать дополнительную информацию в консоль в момент изменения значения максимальной длины ветви дерева;

d. `DebugLevel.SMART_INSERT` – отображать дополнительную информацию в консоль во время применения «интеллектуальной» вставки в дерево (с дополнительной балансировкой).

4. В соответствии с *cprog п.8.5 «Двоичное дерево» (пример 85-03.cpp, листинг 4)* был доработан метод балансировки дерева на языке программирования *Java* (листинг 5).

```
//-----85-03.cpp
//-- обход дерева с сохранением строк в линейном массиве
void set(char *pp[], int n, int &ln){
    if (n>=sz || p[n]==NULL) return;
    set(pp,2*n,ln);
    pp[ln++]=p[n];
    set(pp,2*n+1,ln);
}

// Построение сбалансированного дерева
void balance(char *p[], int a, int b){
    if (a>b) return;
    int m=(a+b)/2; // взять строку из середины интервала
    insert(1,p[m]); // и включить в двоичное дерево
    balance(p,a,m-1); // рекурсивно выполнить для левой и
    balance(p,m+1,b); // правой частей
}

// Балансировка дерева
void balance(){
    int sz1=size(1),ln=0;
    char **pp=new char*[sz1];
    set(pp,1,ln);
    delete p;
    init();
    balance(pp,0,sz1-1);
};
```

Листинг 4. Пример балансировки двоичного дерева в массиве (cprog)

```
//обход дерева с сохранением значений в векторе
private void set(Vector<Object> t, int n){
    if (n>=size || arrayTree.get(n) == null) return;
    set(t,2*n+1);
    t.add(arrayTree.get(n));
}
```



```

        set(t, 2*n+2);
    }

    // Построение сбалансированного дерева
    private void balance(Vector<Object> t, int a, int b,
        ArrayList<Object> r) {
        if (a>b) return;
        if (a==b) return;
        int m=(a+b)/2; // взять строку из середины интервала
        insertRecursive(r, 0, t.get(m));
        balance(t, m+1, b, r); // рекурсивно выполнить для левой и
        balance(t, a, m, r); // правой частей
    }

    //Балансировка дерева
    public BinaryTreeArray balance(){
        int sz1=getSize(0);
        Vector <Object> newArray = new Vector<Object> (size);
        //вектор индексов
        newSizeBalance = 10;
        ArrayList<Object> newArrayTree = new
        ArrayList<Object>(size);
        for(int i = 0; i < size; i++) {
            newArrayTree.add(null);
        }
        set(newArray, 0);
        balance(newArray, 0, sz1, newArrayTree);
        int curMaxLevel = 0;
        if (this.maxLevel != 0)
            if (this.maxLevel == 1)
                curMaxLevel = 1;
            else
                curMaxLevel = log2Count(countObject);
        else
            curMaxLevel = 0;
        BinaryTreeArray balanced = new
        BinaryTreeArray(newSizeBalance, newArrayTree, this.comparator,
        this.countObject, curMaxLevel);
        return balanced;
    }

```

Листинг 5. Метод балансировки в классе структуры данных

5. Изменён метод вставки значения в дерево *addValue* (листинг 6).

В передаваемых ей аргументах появились новые:

- a. *boolean smartBalance* – при значении *true* используется режим с «интеллектуальной» вставкой значения в дерево (с использованием балансировки), при значении *false* – происходит обычная рекурсивная вставка в дерево (без дополнительной балансировки);
- b. *double threshold* – пороговое значение (коэффициент), при котором будет срабатывать «интеллектуальная» вставка (используется, если *smartBalance* принимает значение *true*);
- c. *DebugLevel debug* – значение вывода дополнительной информации в консоль.

```
// Вспомогательный метод вставки значения в массив
private void insertRecursive(int current, Object obj, int
level, boolean smartBalance, double threshold, DebugLevel
debug){
    while (current >= size) {
        if (debug == DebugLevel.INCREASE_ARRAYLIST)
            System.out.println("INCREASE ARRAYLIST.
Before: countObject = " + countObject + "; size = " + size +
"; log2(count) = " + log2Count(countObject) + "; maxlevel = "
+ maxLevel + ".");
        size*=2;
        for (int i = size/2; i <= size; i++) // с обнулением
НОВОЙ части
            arrayTree.add(null);
        if (debug == DebugLevel.INCREASE_ARRAYLIST)
            System.out.println("INCREASE ARRAYLIST. After: new
size = " + size + ".");
    }
    if (arrayTree.get(current) == null) {
        arrayTree.set(current, obj);
        if (maxLevel < level)
        {
            if (debug == DebugLevel.CHANGE_MAXLEVEL)
                System.out.println("CHANGE MAXLEVEL. Before:
maxLevel = " + maxLevel + "; count = " + countObject + ";
log2(count) = " + log2Count(countObject) + ".");
            maxLevel = level;
        }
    }
}
```

```

        if (debug == DebugLevel.CHANGE_MAXLEVEL)
            System.out.println("CHANGE MAXLEVEL. After:
new maxLevel = " + maxLevel + "; log2(count) = " +
log2Count(countObject) + ".");
    }

    if (maxLevel > log2Count(countObject)*threshold &&
smartBalance){
        if (debug == DebugLevel.SMART_INSERT)
            System.out.println("!!!!SMART INSERT!!!!.
Before: countObject = " + countObject + "; size = " + size +
"; maxLevel = " + maxLevel + "; log2Count = " +
log2Count(countObject) + ".");
        BinaryTreeArray btsBalanced = this.balance();
        arrayTree = btsBalanced.getArrayTree();
        maxLevel = btsBalanced.getMaxLevel();
        size = btsBalanced.getSizeArrayTree();
        countObject = btsBalanced.getCountObject();
        if (debug == DebugLevel.SMART_INSERT)
            System.out.println("!!!!SMART INSERT!!!!.
After: countObject = " + countObject + "; size = " + size + ";
maxLevel = " + maxLevel + "; log2Count = " +
log2Count(countObject) + ".");
    }
    return;
}

    if (comparator.compare(obj,arrayTree.get(current)) < 0)
        insertRecursive(2 * current + 1, obj, level+1,
smartBalance, threshold, debug);
    else
        insertRecursive(2 * current + 2, obj, level+1,
smartBalance, threshold, debug);
}

// Вставка значения в дерево
public void addValue(Object value, boolean smartBalance,
double threshold, DebugLevel debug) {
    countObject++;
    insertRecursive(0, value, 0, smartBalance, threshold,
debug);
}

```

Листинг 6. Метод вставки значения в дерево addValue

6. Реализован метод *log2Count*, который возвращает значение максимальной длины ветви в сбалансированном дереве от числа объектов, хранимых в нём (листинг 7).

```
private int log2Count(int count){  
    return (int) (Math.log10(count)/Math.log10(2));  
}
```

Листинг 7. Метод log2Count

В ходе выполнения лабораторной работы возникла проблема с переполнением кучи JVM. Проведём несколько экспериментов, чтобы выяснить возможную причину возникновения этой проблемы, используя разработанные нововведения в тестируемом программном обеспечении.

Эксперимент 1.

Производилась вставка в дерево 100 000 объектов типа *IntegerClass*. Выключен режим «интеллектуальной» вставки. Включен *DebugLevel.INCREASE_ARRAYLIST*. На рис. 21 продемонстрирован результат работы.

```
INCREASE ARRAYLIST. Before: countObject = 3896; size = 20971520; log2(count) = 11; maxlevel = 24.  
INCREASE ARRAYLIST. After: new size = 41943040.  
INCREASE ARRAYLIST. Before: countObject = 3945; size = 41943040; log2(count) = 11; maxlevel = 24.  
INCREASE ARRAYLIST. After: new size = 83886080.  
INCREASE ARRAYLIST. Before: countObject = 6737; size = 83886080; log2(count) = 12; maxlevel = 26.  
INCREASE ARRAYLIST. After: new size = 167772160.  
INCREASE ARRAYLIST. Before: countObject = 6780; size = 167772160; log2(count) = 12; maxlevel = 26.  
INCREASE ARRAYLIST. After: new size = 335544320.  
INCREASE ARRAYLIST. Before: countObject = 6835; size = 335544320; log2(count) = 12; maxlevel = 27.  
Exception in thread "main" java.lang.OutOfMemoryError: Create breakpoint : Java heap space  
    at java.base/java.util.Arrays.copyOf(Arrays.java:3512)  
    at java.base/java.util.Arrays.copyOf(Arrays.java:3481)  
    at java.base/java.util.ArrayList.grow(ArrayList.java:237)
```

Рисунок 21. Результат работы вставки

Как видно из рис. 21 при вставке 6835 элементов максимальная длина ветви дерева достигает значения 27, а размер динамического массива на момент возникновения ошибки составил 3 355 443 320 (после этого размер бы стал 6 710 886 640).

Первым предположением возникновения ошибки переполнения кучи является вырожденность самой длинной ветви дерева, как упорядоченного списка.

Эксперимент 2.

Чтобы исключить возникновение ошибки переполнения кучи посредством самой длинной ветви дерева, как упорядоченного списка, проводится эксперимент с «интеллектуальной» вставкой в дерево (с дополнительной балансировкой) при пороговом коэффициенте *threshold* = 1.5.

Таким образом, производилась вставка в дерево 100000 объектов типа *IntegerClass*. Включен режим «интеллектуальной» вставки с пороговым коэффициентом *threshold* = 1.5. Включен *DebugLevel.SMART_INSERT*. На рис. 22 продемонстрирован результат работы.

```
!!!!SMART INSERT!!!!. Before: countObject = 12534; size = 83886080; maxLevel = 20; log2Count = 13.  
!!!!SMART INSERT!!!!. After: countObject = 12534; size = 83886080; maxLevel = 13; log2Count = 13.  
!!!!SMART INSERT!!!!. Before: countObject = 12625; size = 83886080; maxLevel = 22; log2Count = 13.  
!!!!SMART INSERT!!!!. After: countObject = 12625; size = 167772160; maxLevel = 13; log2Count = 13.  
!!!!SMART INSERT!!!!. Before: countObject = 12647; size = 167772160; maxLevel = 20; log2Count = 13.  
Exception in thread "main" java.lang.OutOfMemoryError: Create breakpoint : Java heap space  
at java.base/java.util.Arrays.copyOf(Arrays.java:3512)  
at java.base/java.util.Arrays.copyOf(Arrays.java:3481)  
at java.base/java.util.ArrayList.grow(ArrayList.java:237)  
at java.base/java.util.ArrayList.grow(ArrayList.java:244)
```

Рисунок 22. Результат работы «интеллектуальной» вставки

Из рис. 22 видно, что результат вставки за счёт её «интеллектуальности» улучшился. Количество вставляемых в дерево объектов возросло в 2 раза. Размер массива на момент возникновения ошибки был равен 167 772 160, а максимальная длина ветви 20.

Данные улучшились, но при этом основная цель – вставка 100 000 объектов – не была достигнута. Проанализировав вышеизложенные результаты, возникло ещё одно предположение – значения типа *IntegerClass* генерировались в диапазоне [-1000;1000], т.е. в дерево в

общем могло вставляться 2001 различное значение, включая ноль. Текущая вставка достигла 12647 элементов, это говорит о том, что в дереве встречается достаточно большое кол-во повторяющихся значений типа *IntegerClass*.

Эксперимент 3.

В данном эксперименте увеличим диапазон значений типа *IntegerClass* – [-1 000 000 000; 1 000 000 000]. Тем самым это обеспечит вставку в дерево 2 000 000 001 различных значений типа *IntegerClass*.

Таким образом, производилась вставка в дерево 100000 объектов типа *IntegerClass*. Включен режим «интеллектуальной» вставки с пороговым коэффициентом *threshold* = 1.5. Включен *DebugLevel.INCREASE_ARRAYLIST*. На рис. 23 продемонстрирован результат работы.

```
INCREASE ARRAYLIST. Before: countObject = 67576; size = 327680; log2(count) = 16; maxlevel = 18.  
INCREASE ARRAYLIST. After: new size = 655360.  
INCREASE ARRAYLIST. Before: countObject = 70169; size = 655360; log2(count) = 16; maxlevel = 18.  
INCREASE ARRAYLIST. After: new size = 1310720.  
INCREASE ARRAYLIST. Before: countObject = 71472; size = 1310720; log2(count) = 16; maxlevel = 19.  
INCREASE ARRAYLIST. After: new size = 2621440.  
INCREASE ARRAYLIST. Before: countObject = 74392; size = 2621440; log2(count) = 16; maxlevel = 20.  
INCREASE ARRAYLIST. After: new size = 5242880.  
INCREASE ARRAYLIST. Before: countObject = 96868; size = 5242880; log2(count) = 16; maxlevel = 21.  
INCREASE ARRAYLIST. After: new size = 10485760.  
Insert is done! count = 100000  
  
Process finished with exit code 0
```

Рисунок 23. Результат работы «интеллектуальной» вставки с изменённым диапазоном типа IntegerClass

Из рис. 23 видно, что основная цель экспериментов достигнута – в дерево вставилось 100 000 объектов. Результаты также заслуживают внимания: размер динамического массива на конец эксперимента равен 10 485 760, максимальная длина ветви дерева равна 21. Также можно заметить, что «интеллектуальная» вставка работает исправно, так как на момент последнего увеличения массива максимальная длина ветви дерева

была меньше, чем произведение максимальной длины ветви сбалансированного дерева (при заданном количестве объектов) и значения порогового коэффициента равному 1.5.

Исходя из вышесказанного, что при конфигурации из последнего эксперимента можно производить тестирование производительности метода балансировки.

Код замеров результатов «грязного» времени (рис. 24) удаления значения (листа) из дерева после балансировки приведён в листинге 8.

```
@org.junit.jupiter.api.Test
public void testBalanced() {
    System.out.println("BALANCED TEST");

    for (int countOfElem = 10000; countOfElem <= 10300000;
countOfElem *= 2) {
        for (int i = 0; i < countOfElem; i++){
            actualBts.addValue(protoType.create(), true, 1.3,
DebugLevel.OFF);
        }
        actualBts = actualBts.balance();

        long startTime = System.nanoTime();
        actualBts.removeNodeByIndex(actualBts.getSizeTree() -
1);
        long endTime = System.nanoTime();
        double timeElapsed = (endTime - startTime) * 1.0 /
1_000_000;
        System.out.println("N = " + countOfElem + ". Time = "
+ timeElapsed + " ms.");
        actualBts = actualBts.newTree();
    }
    System.out.println("-----");
}
```

Листинг 8. Метод тестирования сбалансированности дерева и замеров времени

BALANCED TEST	
N = 10000.	Time = 0.1713 ms.
N = 20000.	Time = 0.3298 ms.
N = 40000.	Time = 0.6767 ms.
N = 80000.	Time = 1.3804 ms.
N = 160000.	Time = 2.6699 ms.
N = 320000.	Time = 5.353 ms.
N = 640000.	Time = 10.7137 ms.
N = 1280000.	Time = 22.1382 ms.
N = 2560000.	Time = 42.1872 ms.

Рисунок 24. Результаты выполнения теста

Таблица 1. Таблица расчётов

N	T1(эксп)	NLog2(N)	T1*(матем)	Отклонение
10000	0,171	132877,124	0,141	21,69%
20000	0,330	285754,248	0,303	8,94%
40000	0,677	611508,495	0,648	4,46%
80000	1,380	1303016,990	1,380	0,00%
160000	2,670	2766033,981	2,930	-8,89%
320000	5,353	5852067,961	6,200	-13,66%
640000	10,714	12344135,923	13,077	-18,07%
1280000	22,138	25968271,846	27,510	-19,53%
2560000	42,187	54496543,692	57,733	-26,93%

График теоретической зависимости и «грязного» времени операций взаимодействия со сбалансированным деревом представлен на рис. 25.

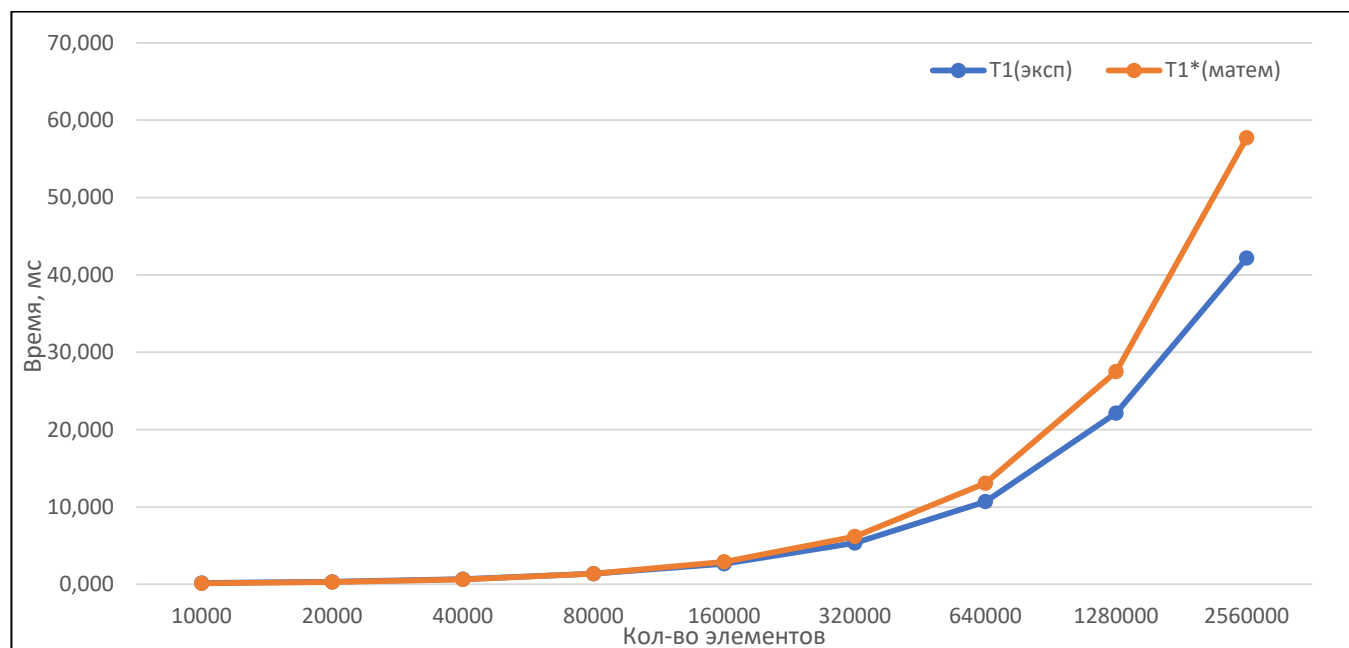


Рисунок 25. График зависимости

Из полученных результатов можно сделать следующий вывод:

Что же касается трудоемкости такого процесса как вставка, поиск, удаление, то она зависит от вида полученного дерева: если оно сбалансировано, то получаем $T=N\log_2 N$, если вырождается в линейный список, то $T=N^2/2$.

«Грязное» время работы программы явно подчиняется логарифмической (теоретической) зависимости. Это заметно, во-первых, по относительно небольшому отклонению, которое по модулю в большинстве случаев не превышает ~27%, а во-вторых, проанализировав рис. 21.

Заключение

В ходе выполнения лабораторной работы было проведено модульное тестирование разработанного метода балансировки класса структуры данных «Бинарное дерево в массиве». Тесты разрабатывались с помощью *JUnit* и составили тестовое покрытие для метода балансировки.

Также был оценён вид зависимости трудоемкости и «грязного» времени работы метода от размерности структуры данных.

Приложение А

Весь исходный код расположен по ссылке: